

# Fondamenti di Informatica

## A.A. 2016-2017

### Prova intermedia del 5 luglio 2017 – Testo 1

NOME \_\_\_\_\_ COGNOME \_\_\_\_\_

-----  
Durante l'esame non si possono usare appunti né altro materiale.  
-----

#### Esercizio 1

Individuare e correggere gli errori nel seguente testo:

Secondo il principio dell'*information hiding*, i dettagli implementativi relativi al funzionamento di una parte di un programma vengono resi disponibili al resto del programma. Un linguaggio di programmazione può fornire meccanismi che consentano al programmatore di definire propri tipi, nascondendone gli aspetti implementativi, secondo il principio dell'*information hiding*. Nel caso di una classe C++, questo si realizza separando la dichiarazione della classe dalla definizione dei suoi metodi, ovvero separando l'interfaccia della classe dalla sua implementazione. In particolare, in C++ la rappresentazione dei dati viene nascosta nella parte pubblica della classe e l'implementazione dei metodi viene realizzata in un file separato rispetto a quello contenente la dichiarazione della classe. Applicando il principio dell'*information hiding*, a determinate condizioni, una modifica all'implementazione della classe non comporta ripercussioni sul resto del programma, finché l'interfaccia pubblica della classe subisce limitate variazioni. Questo approccio è fondamentale per ottenere programmi più robusti, flessibili e riusabili. Il principio dell'*information hiding* è inoltre cruciale nel processo di astrazione dei dati e per la definizione di tipi di dato astratti.

**Soluzione:**

#### Esercizio 2

Date le seguenti dichiarazioni di variabili: `int x; int y; char c; char* p;`  
associare ciascuna delle seguenti istruzioni C++ al suo significato:

|                                      |                                                                         |
|--------------------------------------|-------------------------------------------------------------------------|
| <code>int&amp; r = x;</code>         | Assegnamento dell'indirizzo di una variabile ad un puntatore            |
| <code>double* q = 0;</code>          | Deallocazione della memoria associata a un puntatore                    |
| <code>cout &lt;&lt; *p;</code>       | Operazione logica tra numeri interi                                     |
| <code>void test(int&amp; a);</code>  | Allocazione dinamica della memoria associata a un puntatore             |
| <code>delete p;</code>               | Deallocazione della memoria associata a un puntatore che punta ad array |
| <code>void test(int a);</code>       | Dereferenziazione di una variabile di tipo puntatore                    |
| <code>int b = x &amp;&amp; y;</code> | Definizione con inizializzazione di una variabile di tipo riferimento   |
| <code>p = &amp;c;</code>             | Prototipo di funzione con passaggio per valore                          |
| <code>delete[] p;</code>             | Definizione con inizializzazione di una variabile di tipo puntatore     |
| <code>new p[20];</code>              | Prototipo di funzione con passaggio per riferimento                     |

#### Esercizio 3

Completare le seguenti affermazioni:

1. Per accedere all'oggetto puntato da una variabile puntatore si applica l'operatore di \_\_\_\_\_.
2. I \_\_\_\_\_ forniscono un meccanismo per creare funzioni generiche.
3. Una funzione esterna dichiarata come \_\_\_\_\_ può accedere ai membri privati di una classe.
4. Nelle \_\_\_\_\_ l'ultimo elemento contiene un puntatore al primo elemento.

#### Esercizio 4

Identificare e correggere gli errori nella seguente serie di istruzioni:

```
1 // inserimento di un nodo dopo un altro nodo in una lista semplice
2 template <typename T>
3 void List<T>::insertAfter(Node<T>* p, const T& data) {
4     if (p != 0) {
5         Node<T>* pNew = new Node<T>(data, 0);
6         p->setNext(pNew);
7         pNew->setNext(p->getNext());
8         _numItems--;
9     }
10 }
```

**Soluzione:**

---

**Il seguente esercizio dovrà essere svolto al calcolatore e consegnato su AulaWeb.**

**Accertarsi che la consegna sia confermata dal sistema!**

---

#### Esercizio 5

Allo scopo di rappresentare le matrici diagonali  $n \times n$  in un software per il calcolo scientifico, si realizzi in linguaggio C++ il template di classe *MatDiag*, parametrizzato sul tipo T e avente per attributi la dimensione *\_n* della matrice (un numero intero) e un puntatore *\_p* a un array di *\_n* valori di tipo T, che rappresentano gli elementi sulla diagonale della matrice (gli altri elementi non si rappresentano, avendo valore pari a 0). Si implementino, inoltre, i seguenti metodi:

- Il costruttore di default che inizializzi il valore di *\_n* a zero e il valore di *\_p* a NULL.
- Il costruttore con parametri che riceva come parametri la dimensione *n* della matrice (un numero intero) ed un array D di *n* elementi di tipo T, allochi dinamicamente l'array puntato da *\_p* e vi copi gli elementi di D.
- Il costruttore di copia che allochi dinamicamente l'array puntato da *\_p*, copiando la dimensione dell'array e l'array stesso dall'oggetto passato come parametro.
- Il distruttore.
- I selettori (un metodo per ciascun attributo).
- Il modificatore per l'array puntato da *\_p*. Il modificatore riceverà come parametro un array contenente i nuovi valori. Per semplicità, si supponga che l'array puntato da *\_p* sia già stato correttamente allocato delle dimensioni volute e che l'array passato come parametro al modificatore abbia le dimensioni corrette.
- L'operatore \* che riceva come parametro un valore di tipo T e moltiplichi ciascun elemento della matrice diagonale per tale valore. L'operatore restituirà come valore di ritorno un riferimento a *MatDiag*.
- L'operatore di assegnamento. L'operatore deallocherà l'array puntato da *\_p* e lo riallocherà delle dimensioni opportune, copiandolo dall'oggetto passato come parametro.
- Il metodo *stampa* che stampi a video l'intera matrice diagonale (compresi gli zeri al di fuori della diagonale).

Si sviluppi, infine, la funzione *main* che operi nel modo seguente:

1. Dichiarare un array *v* di 5 numeri reali e chiedere all'utente di inserirne da tastiera gli elementi.
2. Utilizzando il costruttore con parametri, dichiarare un oggetto M di classe *MatDiag*, assegnando il tipo *double* al parametro T. L'oggetto M verrà inizializzato passando al costruttore con parametri della classe *MatDiag* l'array *v* e la sua dimensione (pari a 5).
3. Dichiarare un oggetto P di classe *MatDiag*, assegnando il tipo *double* al parametro T e assegnare M a P.
4. Chiedere all'utente di inserire da tastiera un numero reale *a* e, utilizzando l'operatore \*, moltiplicare P per *a*.
5. Chiamando il metodo *stampa* della classe *MatDiag*, stampare le matrici diagonali M e P e termini.

## Soluzione

### Esercizio 1 – Soluzione (1.5 punti, 0.5 per ogni errore individuato e corretto)

Secondo il principio dell'information hiding, i dettagli implementativi relativi al funzionamento di una parte di un programma **vengono nascosti** al resto del programma. Un linguaggio di programmazione può fornire meccanismi che consentano al programmatore di definire propri tipi, nascondendone gli aspetti implementativi, secondo il principio dell'information hiding. Nel caso di una classe C++, questo si realizza separando la dichiarazione della classe dalla definizione dei suoi metodi, ovvero separando l'interfaccia della classe dalla sua implementazione. In particolare, in C++ la rappresentazione dei dati viene nascosta nella **parte privata** della classe e l'implementazione dei metodi viene realizzata in un file separato rispetto a quello contenente la dichiarazione della classe. Applicando il principio dell'information hiding, a determinate condizioni, una modifica all'implementazione della classe non comporta ripercussioni sul resto del programma, finché l'interfaccia pubblica della classe **resta invariata**. Questo approccio è fondamentale per ottenere programmi più robusti, flessibili e riusabili. Il principio dell'information hiding è inoltre cruciale nel processo di astrazione dei dati e per la definizione di tipi di dato astratti.

### Esercizio 2 – Soluzione (1.5 punti, 0.15 per ogni associazione esatta)

`int& r = x;` -> Definizione con inizializzazione di una variabile di tipo riferimento; `double* q = 0;` -> Definizione con inizializzazione di una variabile di tipo puntatore; `cout << *p;` -> Dereferenziazione di una variabile di tipo puntatore; `void test(int& a);` -> Prototipo di funzione con passaggio per riferimento; `delete p;` -> Deallocazione della memoria associata a un puntatore; `void test(int a);` -> Prototipo di funzione con passaggio per valore; `int b = x && y;` -> Operazione logica tra numeri interi; `p = &c;` -> Assegnamento dell'indirizzo di una variabile ad un puntatore; `delete[] p;` -> Deallocazione della memoria associata a un puntatore che punta ad array; `new p[20];` -> Allocazione dinamica della memoria associata a un puntatore.

### Esercizio 3 – Soluzione (1.0 punto, 0.25 per ogni risposta esatta)

Dereferenziazione (dereference, \*), template di funzioni, funzione amica (friend function), liste circolari semplici.

### Esercizio 4 – Soluzione (1.0 punto, 0.5 per ogni risposta esatta)

Righe 6 e 7 - invertire le righe: `pNew->setNext(p->getNext());`; `p->setNext(pNew);`;

Riga 8 - il numero di elementi deve crescere: `_numItems++`;

### Esercizio 5 – Soluzione (6.0 punti, 1.0 – 0.25 – 0.5 – 0.5 – 0.25 – 0.5 – 0.5 – 0.75 – 0.75 – 0.5 – 0.5)

```
#include <iostream>
using namespace std;

template <typename T>
class MatDiag {
public:
    MatDiag();
    MatDiag(int n, T D[]);
    MatDiag(const MatDiag<T>& M);
    ~MatDiag();
    int getSize() const;
    const T* getData() const;
    void setData(T D[]);
    const MatDiag<T>& operator= (const MatDiag<T>& M);
    MatDiag<T>& operator* (T f);
    void stampa() const;
private:
    int _n; T* _p; };

template <typename T> MatDiag<T>::MatDiag() {
    _n = 0; _p = NULL; }

template <typename T>
MatDiag<T>::MatDiag(int n, T D[]) {
    if (n > 0) {
        _p = new T[n];
        if (_p != NULL) {
            for (int k = 0; k < n; k++)
                _p[k] = D[k];
            _n = n; }
        else _n = 0;
    } else { _n = 0; _p = NULL; } }

template <typename T>
MatDiag<T>::MatDiag(const MatDiag<T>& M) {
    if (M._n > 0) {
        _p = new T[M._n];
        if (_p != NULL) {
            for (int k = 0; k < M._n; k++)
                _p[k] = M._p[k];
            _n = M._n; }
        else _n = 0;
    } else { _n = 0; _p = NULL; } }

template <typename T> MatDiag<T>::~~MatDiag() {
    if (_p != NULL) delete[] _p; }

template <typename T>
int MatDiag<T>::getSize() const {
    return _n; }

template <typename T>
const T* MatDiag<T>::getData() const {
    return _p; }

template <typename T>
void MatDiag<T>::setData(T D[]) {
    for (int k = 0; k < _n; k++)
        _p[k] = D[k]; }

template <typename T>
MatDiag<T>& MatDiag<T>::operator= (const MatDiag<T>& M) {
    if (this != &M) {
        if (_p != NULL)
            delete[] _p;
        if (M._n > 0) {
            _p = new T[M._n];
            if (_p != NULL) {
                for (int k = 0; k < M._n; k++)
                    _p[k] = M._p[k];
                _n = M._n; }
            else _n = 0;
        } else { _n = 0; _p = NULL; } }
    return *this; }

template <typename T>
void MatDiag<T>::stampa() const {
    if (_p != NULL) {
        for (int h = 0; h < _n; h++) {
            for (int k = 0; k < _n; k++)
                if (h != k) cout << "0 ";
            else cout << _p[k] << " ";
            cout << endl; } } }

int main() {
    const int dim = 5 ; double v[dim] = {0.0};
    cout << "Inserire valori: " << endl;
    for (int i = 0; i < dim; i++) {
        cout << "Valore " << i+1 << ": ";
        cin >> v[i]; }
    MatDiag<double> M(dim, v);
    MatDiag<double> P = M;
    double a = 0.0;
    cout << "Inserire un valore: "; cin >> a;
    P * a;
    cout << "M: " << endl; M.stampa();
    cout << "P: " << endl; P.stampa();
    return 0; }
```